گزارش فرانت اند پروژه میخک (سامانه مدیریت یکپارچه خدمات کسب و کار)

۱۴۰۳-۱۴۰۴

فهرست

درباره سامانه ۱
ساختار فایل‌ها ۱
package.json ۱
pnpm-lock.yaml ۱
Dockerfile ۱
docker-compose.yaml ۱
nginx.conf ۱
vite.config.js ۱
.env ۲
.gitigonre ۳
.dockerigonre ۳
./public ۳
./src ۳
./assets ۳
./store ۳
./Components ۳
App.css ۳
Font.jsx ۳
MUITheme.jsx و them.jsx ۳
main.jsx ۳
App.jsx ۴
سامانه به دو بخش کلی تقسیم می‌شود ۵
./Authentication، ./Authorization و ./Base ۵
./ui ۵
./Components/Base/BaseRoute.jsx ۵
RouteAuthentication.jsx ۶
./Authentication ۶
RouteAuthorization.jsx ۷

./Authorization ۷
./Base ۱۰
axios.jsx ۱۰
نمایش toast ۱۰
بررسی دسترسی کاربر ۱۱
تغییر زبان سایت ۱۳

درباره سامانه

سامانه میخک، سامانه مدیریت دسترسی های کاربران و مدیریت سرویس‌های فروش محصولات است. در واقع پنل ادمین این موارد است.

سایت با کتابخانه react و با زبان Javascripts نوشته شده است. باقی موارد که بدیهی است: HTML و CSS. Build tools این پروژه vite هست و از package manager، pnpm استفاده شده است.

از دو Component library به نام های MUI[۱] و Chakraui[۲] استفاده شده است.

توجه شود که حداقل نسخه مورد نیاز Nodejs ۲۲ می‌باشد.

ساختار فایل‌ها

در Root پروژه فایل های زیر قرار دارند:

package.json

پکیج های مورد استفاده در پروژه در این فایل نوشته می‌شوند.

pnpm-lock.yaml

بعد از نصب پکیج ها، شرح dependencyهای مورد نیاز و نصب شده از پکیج‌هایی که در بخش package.json نوشتیم در این فایل آورده می‌شود. در صورتی که این فایل در کنار package.json باشد و دستور pnpm i را بزنیم، dependency ها با همان نسخه ای که در سیستم local نصب و تست شده اند در سیستم دیگر یا سرور نصب خواهند شد.

Dockerfile

پروژه به صورت داکری نوشته شده است. داکرفایل این پروژه می‌گوید پروژه را با استفاده از فایل build.sh، بیلد بگیرد و خروجی نسخه بیلد گرفته شده را در اختیار nginx قرار دهد.

docker-compose.yaml

داکر را به صورت یک ایمیج از داکر کامپوز در می آورد و روی port ۸۰ در دسترس قرار می‌دهد.

nginx.conf

کانفیگ nginx فرانت این پروژه در این فایل قرار دارد. چون پروژه با استفاده از gzip فشرده شده است، برای پشتیبانی از این فشردگی، دستورات پشتیبانی از آن نیز در nginx قرار داده شده است.

vite.config.js

کانفیگ‌های مربوط به vite در این فایل قرار دارند:

---

[۱] برای اطلاعات بیشتر به https://mui.com/material-ui/ رجوع شود.

[۲] برای اطلاعات بیشتر به https://chakra-ui.com/ رجوع شود.

- با استفاده از basicSsl و https: true، پروژه در محیط local با یک گواهی ssl اجرا می‌شود. دلیل آن این است که در محیط توسعه بتوان از طریق بک اندی که در سیستم دیگری در حال اجرا است در سامانه لاگین کرد. چون سامانه minio روی http سوار است و پروژه میخک روی سرور روی https، برای تست امکان اتصال از طریق فرانت به minio، https=true قرار می‌دهیم.
- از دو الگوریتم brotliCompress و gzip برای فشرده سازی استفاده شد.
- برای عیب یابی و مشاهده اینکه چه پکیج‌هایی چه سهمی از سنگینی پروژه را دارند از visualizer استفاده شد.
- برای فعال سازی PWA از VitePWA استفاده شد. تصاویر مورد نیاز برای این بخش در دایرکتوری public قرار دارند.
- برای build گرفتن پروژه نیز کانفیگ‌هایی ست شده:
  - آدرس خروجی از dist به build تغییر کرد.
  - فولدر پیش‌فرض برای فایل‌های استاتیک (CSS, JS, تصاویر و ...) assets/. شد.
  - فشرده‌سازی کدها رو با ابزار سریع esbuild انجام دادیم.
  - هشدار سایز فایل chunk (بر حسب KB) را هم روی ۱۰۰۰ کیلوبایت تنظیم کردیم (پیش‌فرض ۵۰۰ کیلوبایت).
  - در بخش rollupOptions تعریف کردیم که با استفاده از treeshake=true، کدهای استفاده نشده حذف شوند. با استفاده از manualChunks کتابخونه‌های بزرگ رو توی chunkهای جداگانه قرار دادیم تا حجم فایل اصلی کم بشه، cache مرورگر بهتر عمل کنه و لود اولیه سریع‌تر بشه.
- در بخش server:
  - host: true باعث می‌شه سرور dev روی همه اینترفیس‌های شبکه (۰.۰.۰.۰) در دسترس باشه، نه فقط روی localhost.
  - https: true با این کار، سرور dev رو با HTTPS اجرا می‌کنه.
  - strictPort: true با این کار، اگر پورت (اینجا ۳۰۰۰) مشغول باشه، Vite دیگه به پورت بعدی (۳۰۰۱ و ...) سوئیچ نمی‌کنه و خطا می‌ده.
  - در بخش proxy تعریف کردیم که در صورتی که آدرس مورد نظر در فرانت درخواست داده شد، به آدرس دیگری reverse proxy انجام دهد. بخش '/auth/prox/apis/' برای تمامی درخواست های سامانه به backend خود سامانه میخک است و بخش /mikhak/ فقط برای دسترسی مستقیم به سرور minio برای گرفتن عکس‌ها و فایل‌ها است.

.env

متغیرهای مورد نظر که در این پروژه فقط آدرس هایی است که در سامانه از آن استفاده می‌شود تا در بخش proxy یا nginx، reverse proxy انجام شود. با reverse proxy زمانی که سایت روی پروتکل https اجرا شده و نیاز به درخواست به سایت http دارد، بدون خطا درخواست را ارسال می‌کند.

.env.development و .env.production به ترتیب آدرس های مورد نظر برای محیط توسعه و production اند.

.gitigonre

شامل فایل هایی است که نمی‌خواهیم در git ارسال شوند.

.dockerigonre

شامل فایل هایی که نمی خواهیم هنگام ساخت ایمیج داکری وارد داکر شوند.

./public

شامل تصاویر مورد نیاز برای نمایش در tab مرورگر و تصاویر مورد نیاز برای PWA

./src

فایل ها و کامپوننت‌های سامانه در این بخش قرار دارند.

./assets

در این دایرکتوری، فونت های مورد استفاده در پروژه، iconها، عکس ها و lottie فایل ها در این بخش قرار دارند.

./store

مربوط به کانفیگ های Redix هست. تمامی stateهای Redux در این پوشه قرار دارند.

./Components

تمامی کامپوننت‌های سامانه در این دایرکتوری قرار دارند.

App.css

css های مورد مورد استفاده در قسمت های مختلف پروژه در این فایل قرار دارند.

Font.jsx

فونت های سامانه در این بخش تعریف شده اند.

them.jsx و MUITheme.jsx

مربوط به theme های component libraryهای MUI و Chakraui هستند.

main.jsx

مدخل اصلی و اولیه پروژه است.

برای فعال سازی PWA، این بخش از کد را داخل این فایل قرار دادیم:

```
registerSW({ immediate: true });
```

برای استفاده از کامپوننت react-query باید تگ QueryClientProvider> را استفاده کنیم.

برای استفاده از Redux از تگ Provider> استفاده شد.

برای استفاده از Component libraryهای MUI و Chakraui از تگ Provider استفاده شد:

```
export function Provider(props) {
  return (
    <ChakraProvider value={theme}>
      <MUIThemeProvider theme={MUITheme}>
        <ColorModeProvider {...props} />
      </MUIThemeProvider>
    </ChakraProvider>
  )
}
```

در کد بالا مشخص است که base Component library همان Chakraui هست و MUI در مرتبه بعدی قرار دارد. فایل های theme مربوط به هر کدام از این لایبرری ها در این بخش استفاده شده اند.

برای استفاده از react-router نیز از تگ BrowserRouter> استفاده شد.

در این بخش از کد:

```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      retry: 4,
      refetchOnWindowFocus: false,
    },
  },
});
```

تعریف شد که react-query، اگر درخواستی ارسال کرد و با خطا مواجه شد، ۴ بار دیگر با فواصل زمانی کم دوباره همان درخواست را ارسال کند تا اگر مشکل حل شده باشد کاربر با خطا مواجه نشود.

App.jsx

در این فایل routing اصلی پروژه قرار داده شده است. علاوه بر Routing، فونت مورد نظر هم با اضافه کردن تگ Fonts /> به پروژه اضافه شده است و برای پشتیبانی از زبان فارسی نیز از تگ ارائه شده توسط Chakraui به نام LocaleProvider> استفاده شده است. برای نمایش Toast هم نیاز است تا تگ Toaster /> در جایی از پروژه صدا زده شود که در این فایل فراخوانی شد.

برای بررسی Router بررسی فایل ها را به شکل دیگری ادامه بدهیم:

سامانه به دو بخش کلی تقسیم می‌شود

- Authentication: مربوط به بخش ورود، فراموشی رمز عبور، تغییر رمز عبور و ثبت نام است. آدرس فایل های مورد نظر این بخش در src/Components/Authentication/. هست.
- Authorization: مربوط به بخش مدیریت دسترسی کاربران و مدیریت سرویس ها و ... است. آدرس فایل های مورد نظر این بخش در src/Components/Authorization/. هست.

به همین منظور Routing سایت هم به دو بخش تقسیم می‌شود. یکی برای Authentication و یکی هم برای Authorization.

در دایرکتوری src/Components/. ۴ دایرکتوری وجود دارد:

Authentication/.، Authorization/. و Base/.

با توجه به توضیحات بالا مشخص است؛ با این نکته که در صفحات مربوط به بخش Authentication و Authorization، تمام کامپوننت ها و توابعی که مورد نیاز برای این بخش ها باشند در دایرکتوری صفحات همان بخش ها قرار دارند مگر اینکه آن تابع یا کامپوننت در قسمت های مختلف پروژه استفاده شده باشد یا توانایی بالقوه برای استفاده در بخش های مختلف پروژه داشته باشند. در این صورت آن توابع در دایرکتوری Base/. قرار گرفتند.

ui/.

با دستور زیر می توان کامپوننت های مربوط به Chakraui را در پروژه اضافه کرد و با تغییر آن در واقع حالت پیش فرض آن کامپوننت تغییر می‌کند.

```
chakra snippet add button
```

با این کار، کامپوننت مورد در در آدرس component/ui/. ذخیره و در دسترس قرار می‌گیرد.[۳]

Components/Base/BaseRoute.jsx/.

برای Routing، یک فایل به نام src/Components/Base/BaseRoute.jsx/. وجود دارد که آدرس‌های صفحات مورد نظر برای Routing را به صورت متغیر در آن ذخیره کرده ایم.

در بخش App.jsx دیدیم که Routing را در این صفحه تعریف کرده ایم. بر این اساس، با رفتن به آدرس /، به آدرس auth/login/ برای لاگین منتقل می‌شود و در صورت رفتن به آدرس auth/ کامپوننت RouteAuthentication لود شده و در صورت رفتن به آدرس admin/ کامپوننت RouteAuthorization لود می‌شود. در آخر نیز اگر هیچ کدام از این موارد نبود کامپوننت NotFound لود خواهد شد.

---

[۳] برای آشنایی بیشتر به https://chakra-ui.com/docs/get-started/cli رجوع شود.

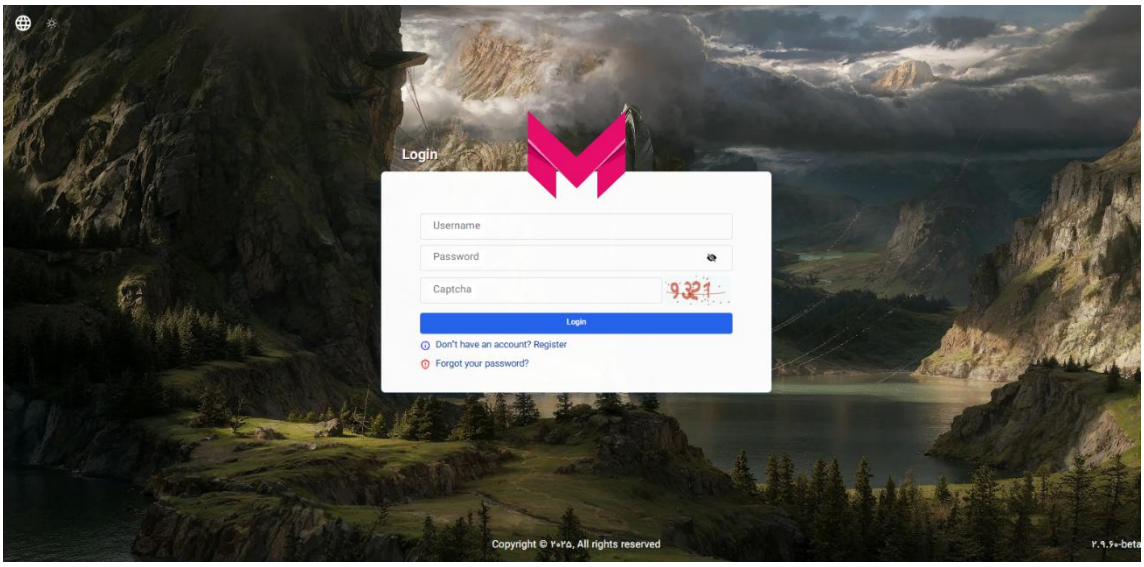
RouteAuthentication.jsx

قالب کلی برای نمایش صفحات مربوط به بخش Authorization به اضافه Routing این بخش در این فایل قرار دارد. Routing این بخش هم همانند چیزی است که در بخش قبل خوانیدم.

./Authentication

```
│   Base.jsx
│   RouteAuthentication.jsx
├──ForgetPassword
│       ForgetPassword.jsx
│       ResetPassword.jsx
├──Login
│       Login.jsx
└──Register
        Register.jsx
        Step1.jsx
        Step2.jsx
        Step3.jsx
```

زمانی که وارد صفحات مربوط به Authentication می‌شویم صفحه ای مانند این می بینیم.



قالب کلی این صفحه در فایل RouteAuthentication.jsx قرار دارد و قالب کلی بخش مرکز و سفید از همین صفحه در فایل Base.jsx

نکته ای که وجود دارد اینکه در صفحه Register، یک صفحه ۳ بخشی داریم که با زدن روی دکمه next، به صفحه بعد می‌رود (در واقع فقط مقدار Base.jsx تغییر می‌کند) و در بخش اول برای بررسی اینکه نام کاربری تکراری به بک‌اند ارسال نکند از متد /user_exist استفاده می‌کند.

کاربر با زدن روی دکمه فراموشی رمز عبور وارد صفحه forget password می شود و در صورتی که ایمیل خود را صحیح وارد کرده باشد بعد از تایید بک اند به صفحه ی reset password برای تغییر نام کاربری خود می رود.

برای نمایش تایم باقی مانده از کتابخانه react-timer-hook استفاده شد.

RouteAuthorization.jsx

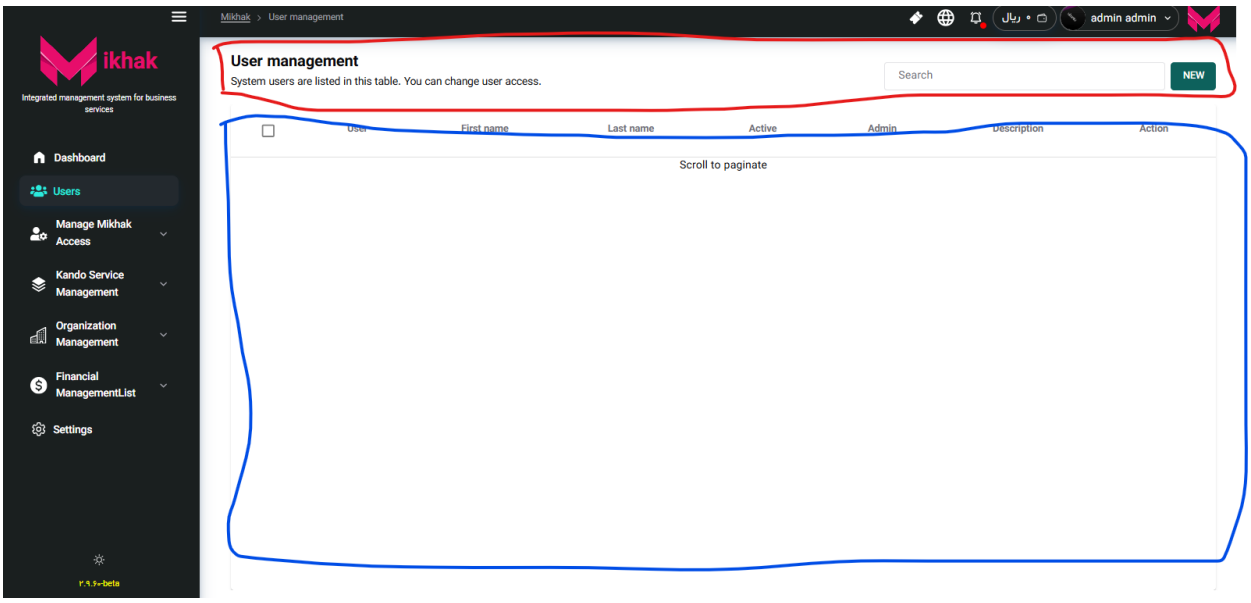
قالب کلی برای نمایش صفحات مربوط به بخش Authorization به اضافه Routing این بخش در این فایل قرار دارد. Routing این بخش هم همانند چیزی است که در بخش قبل خوانیدم.

./Authorization

قالب صفحات مربوط به Authorization در فایل RouteAuthorization.jsx قرار دارد. در این فایل، کامپوننت SideMenu صدا زده شده تا منوی کناری صفحه نمایش داده شود. علاوه بر آن کامپوننت Navbar نیز صدا زده شده تا navbar سایت نمایش داده شود.

نام پوشه ها، هم نام محتوایی است که در بخش بصری فرانت دیده می شود. برای مثال دایکتوری Organization مربوط به بخش organization در بخش SideMenu است.

این صفحات شامل دو بخش اند:



بخش قرمز که در فایل ./pages/BaseHeaderPage.jsx قرار دارد و بخش آبی که مربوط به جدول تمام صفحات است در فایل ./pages/BaseTablePage.jsx قرار دارد.

دایرکتوری های این صفحات به نام همان چیزی است که در SideMenu در بخش بصری سایت می‌بینید. در هر دایرکتوری، اسم همنام همان دایرکتوری مدخل آن صفحه است.

برای ساخت جدول، از یک Custom Hook به نام useTableBaseActions و برای امکان انتخاب سطر با checkbox از یک hook دیگر به نام useCheckboxTable استفاده شده است. تمامی جداول این سایت از جدول قابل مرتب سازی MUI استفاده شده است.

در useTableBaseActions جدول ساخته می‌شود. مرتب سازی جدول مانند کدی است که در سایت MUI[۴] ارائه شده است. با استفاده از useEffect زیر:

```
useEffect(() => {
  const handleKeyDown = (event) => {
    if (event.key === 'Escape') {
      onCloseEscape();
    }
    if (event.altKey && event.key.toLowerCase() === 'n') {
      event.preventDefault();
      onShiftN && onShiftN();
    }
    if (event.altKey && event.key.toLowerCase() === 'a') {
      event.preventDefault();
      onShiftA && onShiftA();
    }
  };

  document.body.addEventListener('keydown', handleKeyDown);
  return () => document.body.removeEventListener('keydown', handleKeyDown);
}, [onCloseEscape, onShiftN, onShiftA]);
```

چند event listener تعریف شد تا کاربر با زدن آنها نتیجه ای در صفحه ببیند. برای مثال با زدن دکمه‌های alt+n مودال Add آن صفحه باز می شود و با زدن esc بسته می شود.

```
const headCells = useMemo(() => {
  let value = [...headCellsValues];

  if (checkAccess) {
    value.push({ id: 'config', label: giveText(4) });
  }

  return value;
}, [checkAccess, headCellsValues]);
```

[4] https://mui.com/material-ui/react-table/#sorting-amp-selecting

headCells برای نمایش هدر جدول است. هدر جدول در این بخش تعریف می شود که اگر دسترسی داشت، بخش config آن هم نمایش داده شود.

برای اینکه pagination به صورت scrolling فعال شود، از تابع useInfiniteQuery از کتابخانه react-query استفاده شد. در تمامی صفحات scroll از بالا به پایین است ولی در صفحه چت تیکت می‌خواهیم از پایین به بالا باشد، به همین دلیل تابع handleScroll نوشته شد تا این موضوع را هندل کند.

برای اینکه react-query بفهمد تا با دیدن آخرین المان، صفحه بعدی را درخواست کند یک تابع به نام lastElementRef نوشته شد تا به صورت ref به آخرین المان داده شود و pagination انجام شود.

برای حذف هر سطر هم از تابع removeAxios استفاده می‌شود.

Custom hook به نام useCheckboxTable نیز همان طور که گفته شد برای کنترل وضعیت فعال یا غیرفعال بودن checkbox های جدول است. در این hook id تمامی سطرهایی که checkboxشان true شده در setیی به نام ids ذخیره می‌شود. برای تغییر وضعیت checkbox هدر جدول به صورت برخی انتخاب شده یا همه انتخاب شده از دو متغیر به نام‌های isAnyChecked و isAllChecked استفاده شد. باقی توابع آن نیز برای خارج کردن یک checkbox از حالت انتخاب شده و یا اضافه کردن آن یا اضافه کردن همه یا حذف همه است.

تمامی دیالوگ های این سایت به صورت lazy استفاده شده اند.

در صفحاتی که بخش checkbox دارند، چیزی به نام ActionBar نیز وجود دارد تا با فعال شدن حداقل یک checkbox، چیزی شبیه به این عکس نمایش داد شود.

باقی صفحات سایت نیز به همین طریق کار می‌کنند. غیر از صفحه services که نمایش آن مورد خاصی ندارد و برای نمایش گرافی آن از کتابخانه React-flow استفاده شده است.

تمامی input ها به صورت Custom component در آدرس ./Base/CustomComponents قرار دارند.

تمامی custom hook ها نیز در آدرس ./Base/CustomHook قرار دارند.

تمامی آیکون های استفاده شده در سایت، در دایرکتوری ./Base/CustomIcons قرار داده شده اند.

صفحه ی Tickets نیز در بخش Navbar سایت قرار دارد؛ فایل های آن نیز در آدرس ./Base/Navbar قرار داده شدند.

./Base

axios.jsx

یکی از مهم ترین بخش های سایت، همین بخش است. در این بخش کانفیگ های مربوط به درخواست به یک اند قرار دارد. در این بخش از کتابخانه axios استفاده شده است.

بک اند سایت به این گونه زده شده است که بعد از لاگین، یک token و refresh token از بک اند به صورت خودکار در کش مرورگر ذخیره می شود. این توکن ها تاریخ انقضا دارند. برای هر درخواست، غیر از درخواست های مربوط به بخش Authentication، باید این توکن به همراه درخواست به بک اند ارسال شود. برای این منظور یک تابع به نام getAccessTokenCookie نوشته شده که می تواند با استفاده از کتابخانه universal-cookie، کوکی های مرورگر را خوانده و مقدار access-tocken آن را برگرداند.

بعد از درخواست، اگر توکن منقضی شده باشد برای دریافت توکن جدید باید ای‌پی‌آی refresh/ را صدا بزنیم و مقدار آن را در کوکی ذخیره کنیم. با این حال اگر چندین درخواست برای ارسال به این تابع فرستاده شده باشند و یکی از آنها متوجه شد که توکن منقضی شده، باید باقی درخواست ها منتظر توکن جدید بمانند و بعد ارسال شوند.

در صورتی که ای‌پی‌آی refresh/ خطای ۴۰۱ بدهد یعنی تاریخ انقضای دوم توکن گذشته و نیاز است تا کاربر دوباره لاگین کند.

در واقع توکن این سایت دو زمانه است یعنی یک زمان تا اینکه کاربر بعد از انقضای اولیه بتواند با صدا زدن refresh/، توکن جدید را بگیرد و یکی برای اینکه فرض کنید فردی در سامانه لاگین کرده و فراموش کرده لاگ اوت کند؛ فردا بر میگردد و فرد دیگری پشت سیستم می نشیند. او نباید بتواند از سامانه بدون لاگین استفاده کند. برای همین زمان دوم تعیین شده تا در صورت استفاده نشدن طولانی مدت از سامانه، کاربر به طور خودکار از آن خارج شود.

تمام این موارد در این فایل لحاظ شده است.

نمایش toast

نمایش toast یکی از مواردی است که در بسیاری از قسمت های پروژه استفاده شده است. برای این منظور چندین تابع نوشته شده است. آدرس این توابع در فایل ./src/Base/BaseFunction.jsx است.

تابع showToast برای نمایش یک toast.

زمانی که یک درخواست ارسال می‌شود برای اینکه ابتدا یک toast به صورت loading نمایش داده شود و سپس یک در صورت دریافت کردن یا نکردن جواب از بک اند، حالت های Successfulll و error و warning یا info را نمایش بدهد از سه تابع زیر استفاده شد:

promiseToast: برای نمایش toast با حالت loading

updatePromiseToastSuccessWarningInfo: برای آپدیت toast زمانی که یا موفقیت آمیز بوده است یا هشدار دریافت کرده یا اینکه می خواهد یک info نمایش دهد.

updatePromiseToastError: برای زمانی که به خطا برخورد کرده است.

دریافت کپچا

برای دریافت کپچا یک تابع در ./src/Components/Base/BaseFunctions.jsx به نام giveMeCaptcha وجود دارد:

```
export const giveMeCaptcha = (dispatch) => {
  fetchWithAxios.get(`/captcha`).then(response => {
    dispatch(setCaptchaImage(response.data.image));
    dispatch(setId(response.data.id));
  });
};
```

علاوه بر این یک کامپوننت به نام ./src/Components/Base/GetCaptcha.jsx وجود دارد که در صورت دریافت یا تغییر کپچا به دلیل استفاده از Redux، مقدار آن به طور خودکار به روز می‌شود.

بررسی دسترسی کاربر

در این سایت تلاش شده تا اگر کاربر به قسمتی از سایت دسترسی نداشت، آن قسمت به کاربر نمایش داده نشود. برای این منظور باید دسترسی های کاربر به api های مختلف را از بک اند می‌گرفتیم. برای

این کار، بک اند، دسترسی های کاربر را به همراه پاسخ ای‌پی‌آی user_info/ برمیگرداند. چیزی شبیه به این:


```
{
      "user_info": {
            "id": 116,
            "username": "admin",
            "email": "admin@gmail.com",
            "name": "admin",
            "family": "admin",
            "description": "admin",
            "disabled": false,
            "admin": true,
            "profile_pic": "/mikhak/…",
            "organizations": []
        },
        "system_user_access": [
            "/services/all_get",
            "/ticket/chat_get",
        ],
}
```


به همین ترتیب admin بودن کاربر با کلید admin و دسترسی های کاربر با کلید system_user_access در redux ذخیره شد. سپس در هر جایی از سایت، با بررسی اینکه آیا کاربر دسترسی مورد نیاز برای دیدن یک کامپوننت را دارد یا خیر، آن کامپوننت برایش نمایش داده می شود یا نمی‌شود.

لیست system_user_access در واقع آدرس api به علاوه متد مورد نظر برای آن api است. برای مثال اگر کاربر بخواهد جدول Role را ببیند باید دسترسی به role/all api/ متد get را داشته باشد؛ پس در این لیست باید مقدار role/all_get/ موجود باشد.

توضیح بیشتر؛ اگر کاربر admin باشد که به تمام اجزای سایت دسترسی دارد. ولی اگر admin نباشد و برای مثال، لیست system_user_access شامل user/all_get/ باشد و کاربر بخواهد صفحه user management را ببیند با این شرط، دسترسی آن بررسی می شود:

```
If (accessSlice.isAdmin || accessSlice.userAccess?.includes(GET_ALL_USER)) {
```

تمامی آدرس apiهای سایت در ./src/Components/Base/UserAccessNames.jsx. به صورت متغیر ذخیره شده اند.

تغییر زبان سایت

تغییر زبان در سایت بسیار راحت است. فایل کلمات و جملات فارسی و انگلیسی در این آدرس ایجاد شده است: ./src/Comonents/Multilanguages/languages/Texts…..json.

انواع زبان ها هم در فایل کناری آن یعنی Names.jsx ذخیره شدند.

توابع مورد نیاز آنها نیز در یک دایرکتوری قبل تر به نام HandleLanguage.jsx هندل می شوند.

زبان فعلی در localstorage با کلید lang ذخیره می‌شود.

با استفاده از تابع giveText با توجه به زبان فعلی، متن مورد نظر برگردانده می شود. برای مثال در یک تگ Text> اگر این تابع را صدا بزنیم و شماره index متن مورد نظر را بفرستیم، دنبال آن index در فایل json می گردد و متن مورد نظر را برمیگرداند.

در برخی از مواقع نیاز داریم تا direction زبان خود را بدانیم تا direction تگ را بر این اساس تنظیم کنیم برای این کار از تابع giveDir استفاده شد. این تابع یک ورودی هم می گیرد تا اگر true بود مقدار را برعکس برگرداند (این برعکس بودن در نمایش Grid کمک می‌کند).

برخی از api ها پاسخی شبیه زیر دارند:

```
{
    fa: "خطای رخ داده است",
    en: "An Error occured"
}
```

در این صورت اگر بخواهیم بر اساس زبان کاربر متن فارسی یا انگلیسی را نمایش دهیم از تابع giveTextBaseInput استفاده می کنیم که ورودی آن همان json بالا است و خروجی آن متن مورد نظر بر اساس زبان فعلی سایت.

برای اینکه اگر به صفحه ای وارد شدیم و دکمه آن در SideMenu رنگی شود، از نام های موجود در ./src/Components/Base/PageNames.jsx. استفاده می کنیم. به این صورت که این نام را هنگام صدا زدن کامپوننت همان صفحه در stateیی از redux می ریزیم و هنگام ساخت SideMenu آن را چک می کنیم.